

AI ASSISTED WEEK 15.4

T. VIKAS

2303A52281

BATCH 43

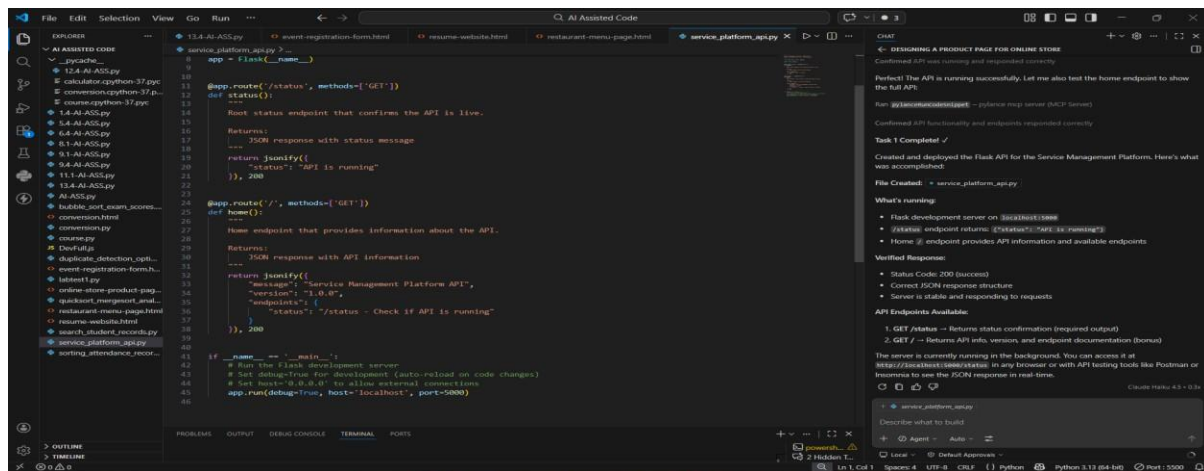
TASK-1:

Prompt: Initial API Setup for a Service Platform

Scenario

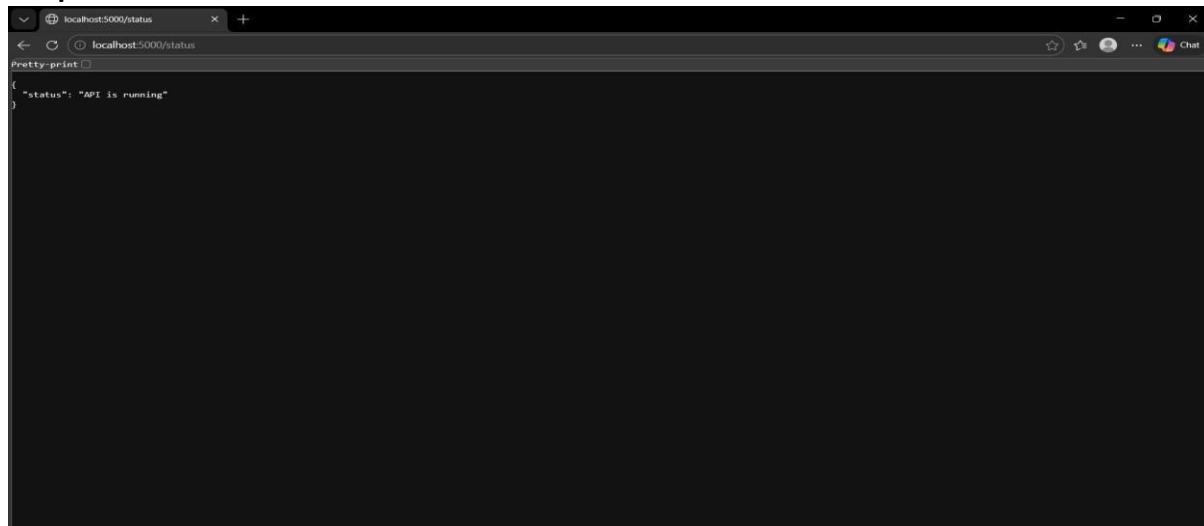
You are developing the backend for a service management platform and need a basic API to confirm that the backend is running.

Code:



```
1  # service_platform_api.py
2  from flask import Flask
3
4  app = Flask(__name__)
5
6  @app.route("/status", methods=['GET'])
7  def status():
8      """
9      Root status endpoint that confirms the API is live.
10     """
11     Returns:
12     ---
13     JSON response with status message
14     ---
15     return jsonify({
16         "status": "API is running"
17     })
18
19 @app.route("/", methods=['GET'])
20 def home():
21     """
22     Home endpoint that provides information about the API.
23     """
24     Returns:
25     ---
26     JSON response with API information
27     ---
28     return jsonify({
29         "message": "Service Management Platform API",
30         "version": "1.0.0",
31         "endpoints": [
32             {
33                 "status": "/status - Check if API is running"
34             }
35         ]
36     })
37
38 if __name__ == '__main__':
39     # To run the flask development server
40     # Set debug=True for development (auto-reload on code changes)
41     # Set host=0.0.0.0 to allow external connections
42     app.run(debug=True, host='localhost', port=5000)
```

Output:



```
{
  "status": "API is running"
}
```

Observation:

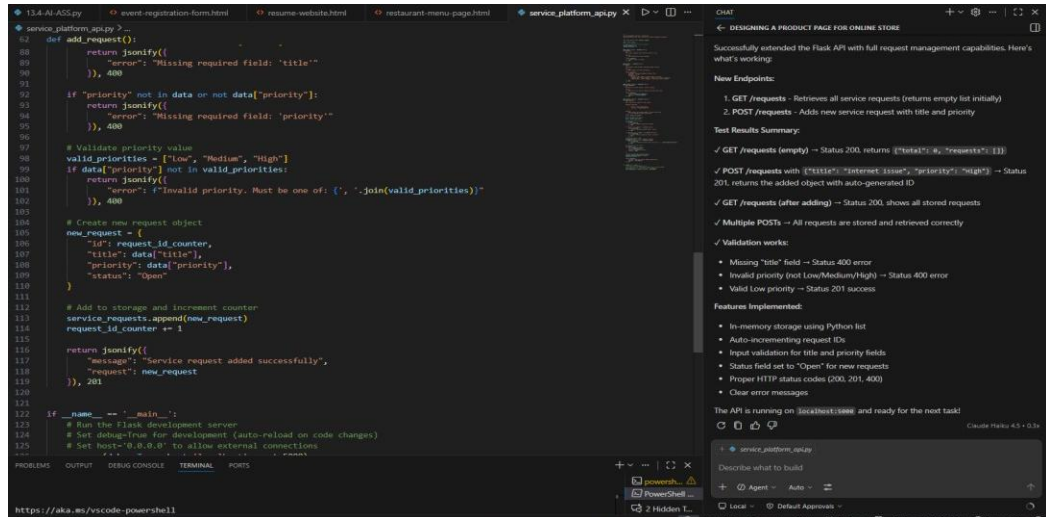
Using GitHub Copilot helped quickly generate a basic Flask setup with minimal configuration. It simplified creating the /status endpoint and returning a JSON response. Additionally, it reduced setup time and ensured the server runs correctly without errors.

TASK-2:

Prompt: Create API to Add and View Resources

Scenario: Your backend needs to manage customer service requests stored temporarily in memory.

Code:



```
134 AI-ASS.py  event-registration-form.html  resume-website.html  restaurant-menu-page.html  service_platform_api.py  CHAT
service_platform_api.py
82 def add_request():
83     return jsonify({
84         "error": "Missing required field: 'title'"
85     }), 400
86
87 if "priority" not in data or not data["priority"]:
88     return jsonify({
89         "error": "Missing required field: 'priority'"
90     }), 400
91
92 # Validate priority value
93 valid_priorities = ["Low", "Medium", "High"]
94 if data["priority"] not in valid_priorities:
95     return jsonify({
96         "error": f"Invalid priority. Must be one of: {', '.join(valid_priorities)}"
97     }), 400
98
99 # Create new request object
100 new_request = {
101     "id": request_id_counter,
102     "title": data["title"],
103     "priority": data["priority"],
104     "status": "Open"
105 }
106
107 # Add to storage and increment counter
108 service_requests.append(new_request)
109 request_id_counter += 1
110
111 return jsonify({
112     "message": "Service request added successfully",
113     "request": new_request
114 }), 201
115
116 if __name__ == "__main__":
117     # Run the flask development server
118     # Set debug=True for development (auto-reload on code changes)
119     # Set host="0.0.0.0" to allow external connections
120     app.run(debug=True, host="0.0.0.0")
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
```

DESIGNING A PRODUCT PAGE FOR ONLINE STORE

Successfully extended the Flask API with full request management capabilities. Here's what's working:

New Endpoints:

- GET /requests - Retrieves all service requests (returns empty list initially)
- POST /requests - Adds new service request with title and priority

Test Results Summary:

- ✓ GET /requests (empty) → Status 200, returns [{"title": "New Request"}]
- ✓ POST /requests with {"title": "New Request", "priority": "High"} → Status 201, returns the added object with auto-generated ID
- ✓ GET /requests (after adding) → Status 200, shows all stored requests
- ✓ Multiple POSTs → All requests are stored and retrieved correctly

Validation works:

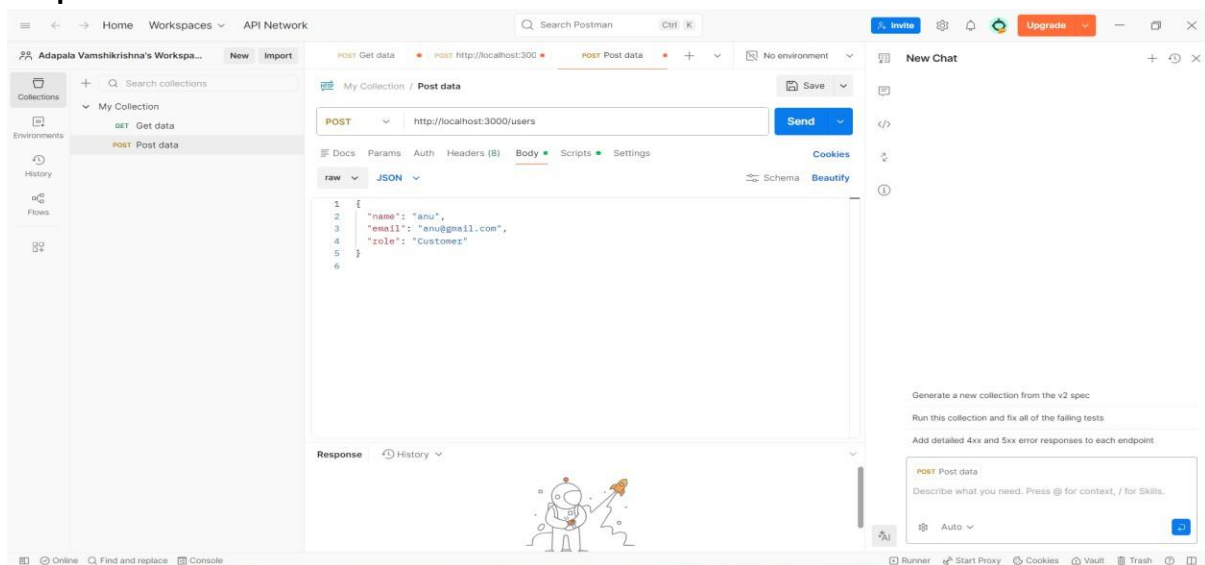
- Missing "title" field → Status 400 error
- Invalid priority (not Low/Medium/High) → Status 400 error
- Valid Low priority → Status 201 success

Features Implemented:

- In-memory storage using Python list
- Auto-incrementing request IDs
- Input validation for title and priority fields
- Status field set to "Open" for new requests
- Proper HTTP status codes (200, 201, 400)
- Clear error messages

The API is running on 0.0.0.0:5000 and ready for the next task!

Output:



Observation:

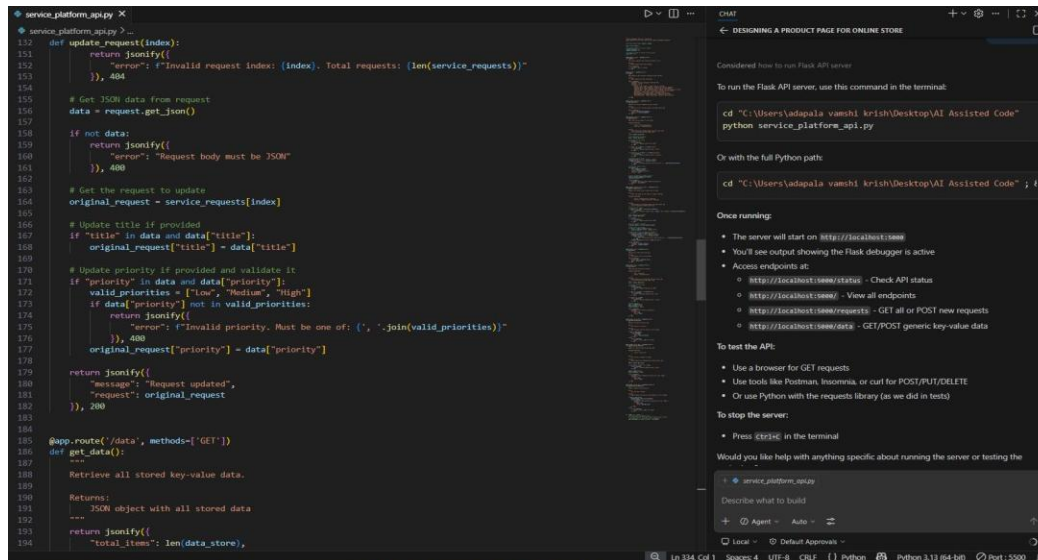
Copilot not only sped up endpoint creation but also helped maintain consistent API structure and naming conventions. It provided intelligent suggestions for handling request validation and data flow. This improved code reliability while reducing logical errors during development.

TASK-3:

Prompt: Modify Existing Resource Using AI Assistance

Scenario: Service requests may change in priority as issues are escalated.

Code:



```
def update_request(index):
    return jsonify({
        "error": f"Invalid request index: {index}. Total requests: {len(service_requests)}"
    }), 404

# Get JSON data from request
data = request.get_json()

if not data:
    return jsonify({
        "error": "Request body must be JSON"
    }), 400

# Get the request to update
original_request = service_requests[index]

# Update title if provided
if "title" in data and data["title"]:
    original_request["title"] = data["title"]

# Update priority if provided and validate it
if "priority" in data and data["priority"]:
    valid_priorities = ["Low", "Medium", "High"]
    if data["priority"] not in valid_priorities:
        return jsonify({
            "error": f"Invalid priority. Must be one of: {', '.join(valid_priorities)}"
        }), 400
    original_request["priority"] = data["priority"]

return jsonify({
    "message": "Request updated",
    "request": original_request
}), 200

@app.route('/data', methods=['GET'])
def get_data():
    """
    Retrieve all stored key-value data.

    Returns:
    - JSON object with all stored data
    """
    return jsonify({
        "total_items": len(data_store),
    })
```

CHAT

DESIGNING A PRODUCT PAGE FOR ONLINE STORE

Considered how to run Flask API server

To run the Flask API server, use this command in the terminal:

```
cd "C:\Users\adapala.vamshi.krish\Desktop\AI Assisted Code"
python service_platform_api.py
```

Or with the full Python path:

```
cd "C:\Users\adapala.vamshi.krish\Desktop\AI Assisted Code" ; python service_platform_api.py
```

Once running:

- The server will start on <http://localhost:5000>
- You'll see output showing the Flask debugger is active
- Access endpoints at:
 - <http://localhost:5000/status> - Check API status
 - <http://localhost:5000/> - View all endpoints
 - <http://localhost:5000/requests> - GET all or POST new requests
 - <http://localhost:5000/data> - GET/POST generic key-value data

To test the API:

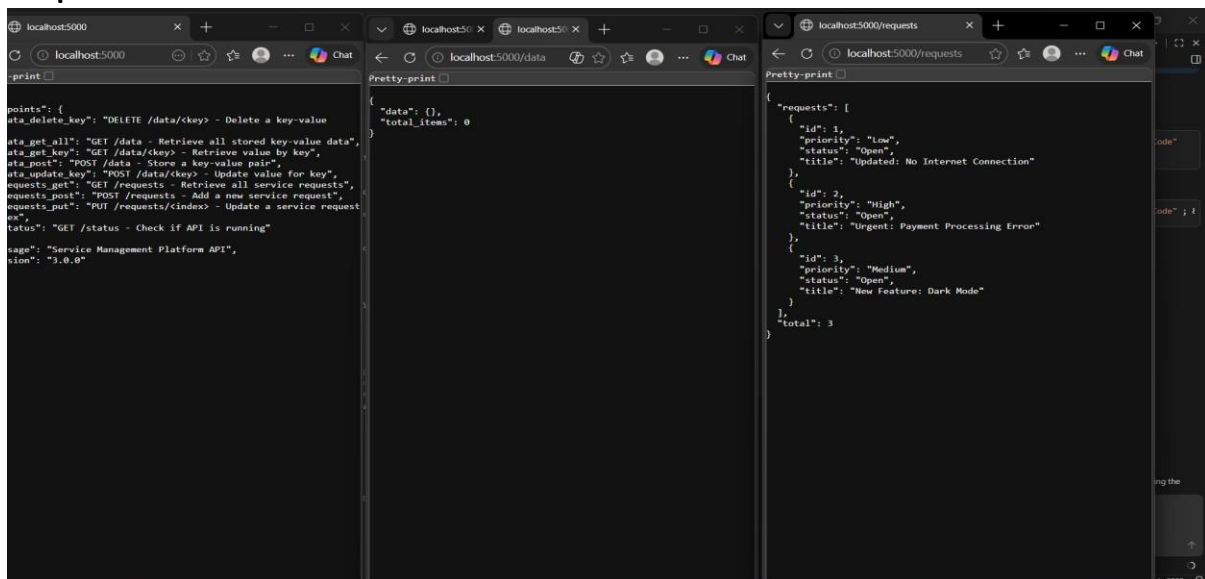
- Use a browser for GET requests
- Use tools like Postman, Insomnia, or curl for POST/PUT/DELETE
- Or use Python with the requests library (as we did in tests)

To stop the server:

- Press `CTRL+C` in the terminal

Would you like help with anything specific about running the server or testing the

Output:



```
points: {
  ata_delete_key: "DELETE /data/key> - Delete a key-value",
  ata_get_all: "GET /data - Retrieve all stored key-value data",
  ata_get_key: "GET /data/key> - Retrieve value by key",
  ata_post: "POST /data - Store a key-value pair",
  ata_update_key: "POST /data/key> - Update value for key",
  requests_get: "GET /requests - Retrieve all service requests",
  requests_post: "POST /requests - Add a new service request",
  requests_put: "PUT /requests/index> - Update a service request",
  status: "GET /status - Check if API is running"
}

page: "Service Management Platform API",
version: "3.0.0"
```

```
{
  "data": {},
  "total_items": 0
}
```

```
{
  "requests": [
    {
      "id": 1,
      "priority": "Low",
      "status": "Open",
      "title": "Updated: No Internet Connection"
    },
    {
      "id": 2,
      "priority": "High",
      "status": "Open",
      "title": "Urgent: Payment Processing Error"
    },
    {
      "id": 3,
      "priority": "Medium",
      "status": "Open",
      "title": "New Feature: Dark Mode"
    }
  ],
  "total": 3
}
```

Observation:

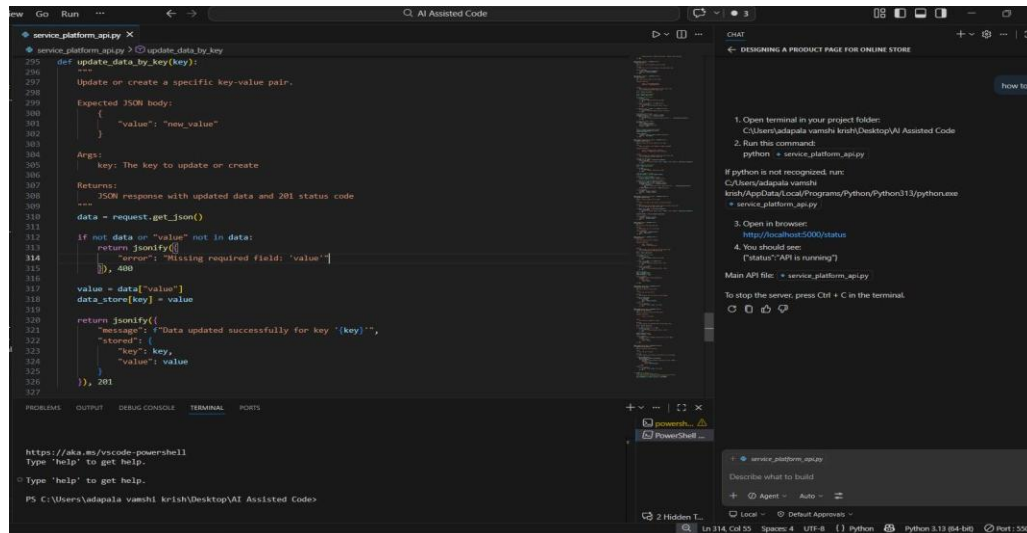
Using GitHub Copilot helped efficiently generate the PUT endpoint with proper update logic. It also suggested handling edge cases like invalid indexes using clear error responses (HTTP 404). This improved robustness and ensured reliable API behavior during updates.

TASK-4:

Prompt: Remove a Resource Safely

Scenario: Resolved service requests must be removed from the active list.

Code:



```
def update_data_by_key(key):
    """
    Update or create a specific key-value pair.

    Expected JSON body:
    {
        "value": "new_value"
    }

    Args:
        key: The key to update or create

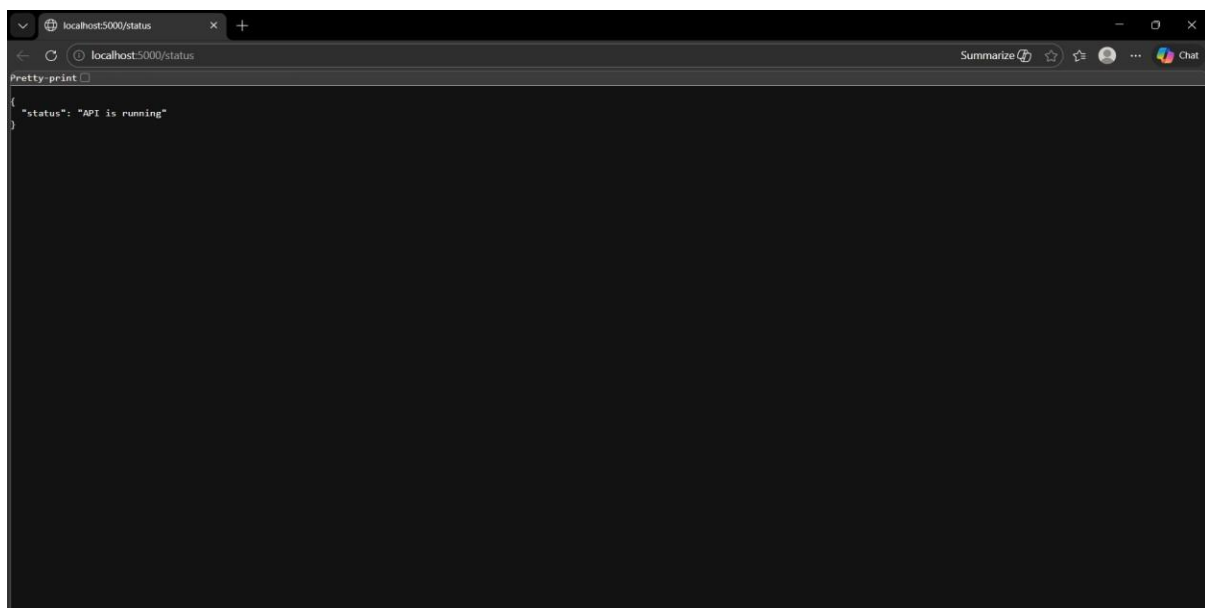
    Returns:
        JSON response with updated data and 201 status code
    """
    data = request.get_json()

    if not data or "value" not in data:
        return jsonify({
            "error": "Missing required field: 'value'"
        }), 400

    value = data["value"]
    data_store[key] = value

    return jsonify({
        "message": "Data updated successfully for key '{key}'",
        "stored": {
            "key": key,
            "value": value
        }
    }), 201
```

Output:



```
{
  "status": "API is running"
}
```

Observation:

Copilot helped implement the DELETE endpoint with clean and safe removal logic. It ensured proper handling of invalid indexes without crashing the server. Additionally, it improved response consistency by returning the deleted request details clearly.

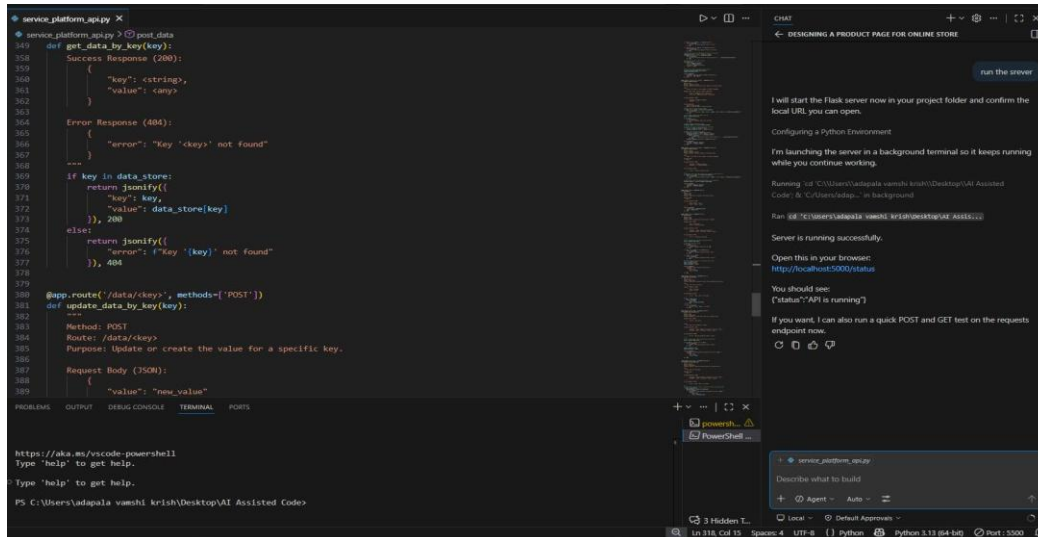
TASK-5:

Prompt: AI-Assisted API Documentation and Readability

Scenario

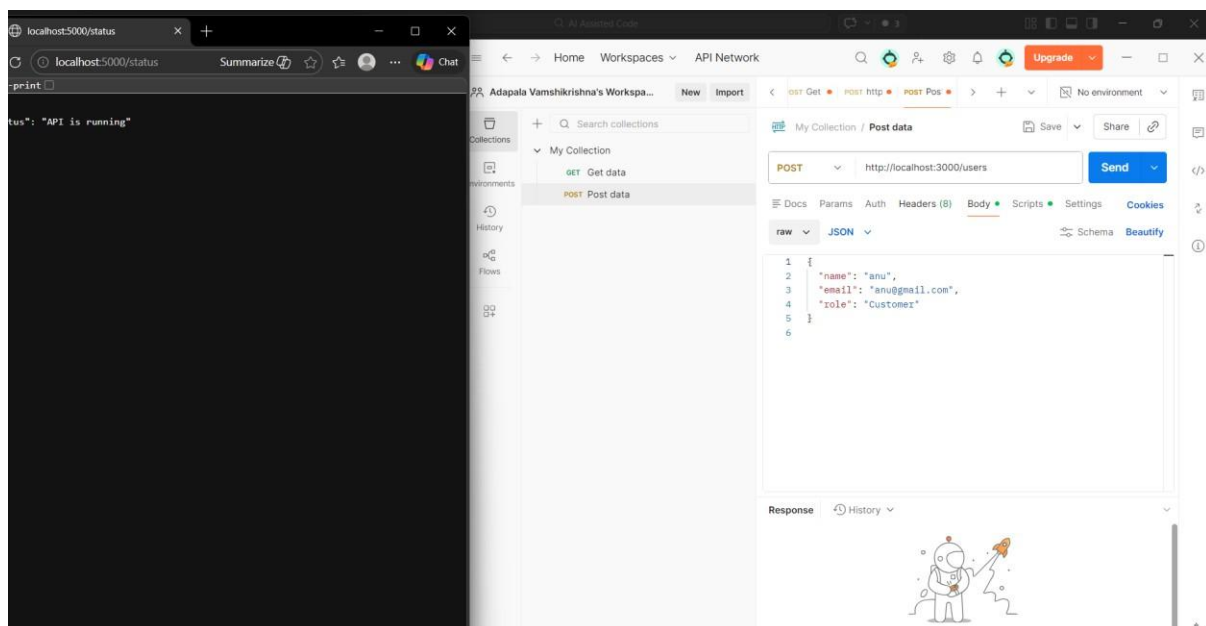
Another developer needs to understand and use your API without reading the full code.

Code:



The screenshot shows a VS Code editor with a Python file named `service_platform_api.py`. The code defines two endpoints: `get_data_by_key` and `update_data_by_key`. The `get_data_by_key` function returns a success response (200) with a JSON object containing a key and its value, or an error response (404) if the key is not found. The `update_data_by_key` function updates the value for a specific key in the `data_store`. The AI chat window on the right provides instructions on how to run the server and test the endpoints. It suggests running the server in a background terminal and provides the URL `http://localhost:5000/status` to open in a browser. It also suggests running a quick POST and GET test on the requests endpoint.

Output:



Observation:

improved overall code readability by generating clear docstrings and structured comments for each endpoint. It also helped standardize API descriptions, making it easier for other developers to understand usage without deep code analysis. Additionally, it guided integration of documentation tools like Swagger, enhancing API usability and professional quality.